

Lecture 1.

**Algorithm complexity estimation.
Linear algorithms.**

**Basics of Algorithms
Ivan Solomatin
MIPT**

Course description

Course description

Course goal:

- Get familiar with basic algorithms and data structures.
- Learn how to implement basic algorithms and data structures on python.
- Learn how to apply obtained knowledge in practice.
- Get experience and intuition in programming problems solving.

Sources:

- Thomas H. Cormen, et al. *Introduction to algorithms*. MIT press, 2009.
- www.geeksforgeeks.org/fundamentals-of-algorithms
- www.e-maxx.ru/algo/

Course description

Course content:

- Lectures (online)
- Homework (problem solving in contest.yandex.ru)
- Seminars (online, real time):
 - Brief repetition of lecture materials
 - Homework analysis
 - Additional topics / problems
 - Q&A

Final grade:

- Homework (~60%)
- Practical exam (problem solving) (~20%)
- Theoretical exam (~20%)

Important requirement:

- To pass the course, you should be able to explain **all** the code you submit to the testing system.

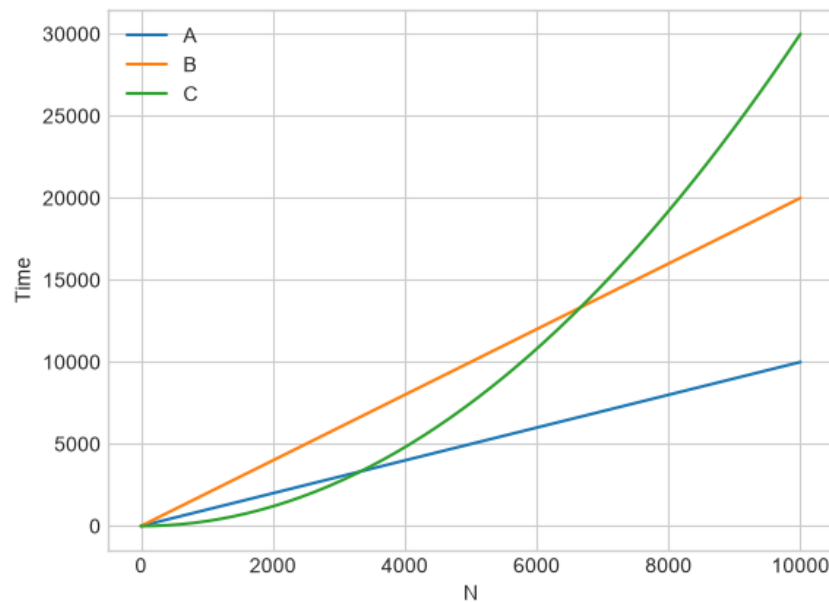
Outline

- Course description.
- Algorithm complexity basics
- Linear algorithms
 - Linear search
 - Invariants
 - Two pointers

Algorithm complexity basics

Algorithm complexity basics

E.g. you have a database containing N items.
You to solve some task using this items as input.
You may use several algorithms: A, B, C.



Which one would you prefer?

Algorithm complexity basics

Definition

Algorithm: a finite sequence of well-defined, computer-implementable instructions, which solves a class of problems.

$$A: X \rightarrow Y$$

Number of elementary operations (processor instructions) for executing A on input $x \in X$:

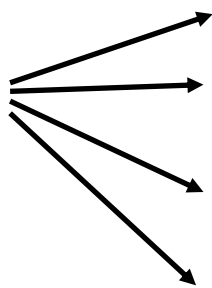
$$t(A, x)$$

Size of input x :

$$s(x)$$

Worst case complexity:

$$N \rightarrow \infty$$

$$T(A, N) = \max_{\substack{x \in X \\ s(x)=N}} t(A, x)$$


- $50N + 100500 = \mathbf{O}(N)$
- $10N^2 + 5N + 1 = \mathbf{O}(N^2)$
- $N \log_2 N = N \frac{\log_c N}{\log_c 2} = \mathbf{O}(N \log N)$
- $42 = \mathbf{O}(1)$

Algorithm complexity basics

Examples

Problem:

Calculate sum of two given numbers x, y :
 $x, y \in [-2^{63}; 2^{63})$

Input:

- x – 64 bit integer = 8 bytes
- y – 64 bit integer = 8 bytes

input size: 16 bytes

Algorithm:

calculate sum, return result.

Complexity:

$O(1)$

Algorithm complexity basics

Examples

Problem:

Calculate sum of N numbers $x_1, \dots, x_N$.
 $x_i \in [-2^{31}; 2^{31})$

Input:

- x_1 — 32bit integer = 4 bytes
- ...
- x_N — 32bit integer = 4 bytes

size: $4N$ bytes

Algorithm:

Iterate over x_i and accumulate sum:

```
res = sum(x)
```

```
res = 0  
for i in range(N):  
    res += x[i]
```

Complexity:

$$O(1) * N = O(N)$$

Algorithm complexity basics

Examples

Problem:

Check if there is a pair of equal numbers between given

$$N \text{ numbers } x_1, \dots, x_N.$$
$$x_i \in [-2^{31}; 2^{31})$$

Input:

- x_1 — 32bit integer = 4 bytes
- ...
- x_N — 32bit integer = 4 bytes

size: $4N$ bytes

Algorithm:

Iterate over each pair and check equality:

Complexity:

$$\text{In worst case: } (N - 1) + (N - 2) + \dots + 1 = \frac{N(N-1)}{2} = O(N^2)$$

```
def f(x):  
    N = len(x)  
    for i in range(N):  
        for j in range(i + 1, N):  
            if x[i] == x[j]:  
                return True  
    return False
```

Linear algorithms

Linear algorithms

- **Linear search**
- Invariants
- Two pointers

Linear Search

	0	1	2	3	4	5	6	7
a	2	3	5	3	0	0	1	8

Is there value “1” in this array?

```
if 1 in a:
```

This is a python list (we can take value by index: `a[i]`).

```
    print('Yes!')
```

So, we can iterate over all possible `i` and check each value to be equal to `1`:

```
else:
```

```
    print('No')
```

```
has_1 = False
```

```
for i in range(len(a)):
```

```
    if a[i] == 1:
```

```
        has_1 = True
```

```
if has_1:
```

```
    print('Yes')
```

```
else:
```

```
    print('No')
```

Linear Search

	0	1	2	3	4	5	6	7
a	2	3	5	3	0	0	1	8
	=1?	=1?	=1?	=1?	=1?	=1?	=1?	=1?
	No	No	No	No	No	No	Yes	No

has_1 = ~~False~~

```
has_1 = False
for i in range(len(a)):
    if a[i] == 1:
        has_1 = True
if has_1:
    print('Yes')
else:
    print('No')
```

Linear Search

	0	1	2	3	4	5	6	7
a	2	3	5	3	0	0	1	8

How do we do that in practice on python?

```
if 1 in a:  
    print('Yes!')  
else:  
    print('No')
```

Algorithmically same thing happens inside.

Operator `in` makes linear search for list, so computational complexity of this operation is $O(N)$.

N.B.: In practice, this operation would be much faster than for loop on previous slide, but it is still $O(N)$.

Linear Search: count

	0	1	2	3	4	5	6	7
a	2	3	1	3	1	0	1	8

How many “1”s in this array?

We iterate over all possible *i*, check each value to be equal to 1:

```
count = 0
for i in range(len(a)):
    if a[i] == 1:
        count += 1
```

Linear Search: count

	0	1	2	3	4	5	6	7	
a	2	3	1	3	1	0	1	8	
	=1?	=1?	=1?	=1?	=1?	=1?	=1?	=1?	count = 0
	No	No	Yes	No	Yes	No	Yes	No	

```
count = 0
for i in range(len(a)):
    if a[i] == 1:
        count += 1
```

Linear Search: count

	0	1	2	3	4	5	6	7
a	2	3	1	3	1	0	1	8

Pythonic way to do the same:

```
count = sum([v == 1 for v in a])
```

Inside, it makes the same (but faster):

```
count = 0
for i in range(len(a)):
    if a[i] == 1:
        count += 1
```

Linear Search: min/max

	0	1	2	3	4	5	6	7
a	2	3	1	3	1	0	1	8

What is minimum(maximum) value in a?

min_value = `min(a)`

We iterate over all possible *i*, keep current min value and check each value to be new minimum:

```
min_value = ∞  
for i in range(len(a)):  
    if a[i] < min_value:  
        min_value = a[i]
```

Linear Search: min/max

	0	1	2	3	4	5	6	7
a	2	3	1	3	1	0	1	8
	<inf?	<2?	<2?	<1?	<1?	<1?	<0?	<0?
	Yes	No	Yes	No	No	Yes	No	No

min = 0

```
min_value = ∞
for i in range(len(a)):
    if a[i] < min_value:
        min_value = a[i]
```

Linear Search: min/max

	0	1	2	3	4	5	6	7
a	2	3	1	3	1	0	1	8

Pythonic way to do the same:

```
min_value = min(a)
```

Inside, it makes the same (but faster):

```
min_value = ∞
```

```
for i in range(len(a)):
```

```
    if a[i] < min_value:
```

```
        min_value = a[i]
```

Linear algorithms

- Linear search
- **Invariants**
- Two pointers

Linear algorithms. Invariants

Sometimes, linear algorithms are not so obvious as in previous cases. And we want to make sure they are correct. You may use idea of invariants to do that.

You define some *invariant* – condition which should always be true. You show that it is true initially, then you show that it remains true after each step of the algorithm. When algorithm finishes you still may guarantee it to be true.

```

min_value = inf
for i in range(len(a)):
    if a[i] < min_value:
        min_value = a[i]

```

Linear algorithms. Invariants

	0	1	2	3	4	5	6	7	
a	2	3	1	3	1	0	1	8	
	<inf?	<2?	<2?	<1?	<1?	<1?	<0?	<0?	min = 0
	Yes	No	Yes	No	No	Yes	No	No	

Let us look at min/max linear search algorithm more formally.

We define pointer (index) i .

And we keep the following *invariant* to be true:

$$\text{min_val} = \min(a[:i])$$

Everything to the right of i is supposed to be unprocessed yet.

- Initially, $i=0$. Strictly saying, $a[:i]$ is empty. Let $\text{min_val} = \infty$.
- When we increase i , *invariant* may become incorrect. We should restore it.

There are two options:

- $a[i] \geq \text{min_val}$ (invariant is still correct)
 - $a[i] < \text{min_val}$ (invariant should be updated: $\text{min_val} = a[i]$).
- Finally, when $i=N$, $\text{min_val} = \min(a)$, so, that is answer to initial problem.

```
sum_value = 0
for i in range(len(a)):
    sum_value += a[i]
```

Linear algorithms. Invariants

	0	1	2	3	4	5	6	7	
a	2	3	1	3	1	0	1	8	sum = 20

If we want to calculate sum of values in the array, we also can do that using invariants.

Invariant:

$\text{sum_value} = \text{sum}(a[:i])$

Everything to the right of i is supposed to be unprocessed yet.

- Initially, $i=0$, $a[:i]$ is empty. $\text{sum_value} = 0$.
- When we increase i , *invariant* becomes incorrect. We restore it:
 $\text{sum_value} += a[i]$
- Finally, when $i=N$, $\text{sum_value} = \text{sum}(a)$, so, that is answer to initial problem.

Linear algorithms

- Linear Search
- Invariants
- **Two pointers**

Linear algorithms. Two pointers

Idea

In linear search we use a pointer: i and we define invariant for this pointer.

Let's continue this idea. Let's use two pointers.

That allows to solve wider class of problems.

Linear algorithms. Two pointers

Pair of values with given sum

	0	1	2	3	4	5	6	7
a	1	2	5	6	8	10	12	15
	≤	≤	≤	≤	≤	≤	≤	

Given sorted array. We should find pair of elements (i, j) , which sum up to given X :

$$a[i] + a[j] = X$$

Naïve solution is to iterate over all pairs of elements:

```
for i in range(N):  
    for j in range(i, N):  
        if a[i] + a[j] == X:  
            return i, j
```

Complexity will be $O(N^2)$. How can we do it faster?

Linear algorithms. Two pointers

Pair of values with given sum

	0	1	2	3	4	5	6	7	
a	1	2	5	6	8	10	12	15	$X = 15$
	$\leq$	$\leq$	$\leq$	$\leq$	$\leq$	$\leq$	$\leq$		

Let us use two pointers: i, j .

Invariant: If desired elements exist, they are inside region: $[i, j]$.

Initially, $i = 0, j = N - 1$. Region $[i, j]$ is whole array.

Then, if $a[i] + a[j] < X$, we can get rid of i :

$$\forall j' \in [i, j]: a[i] + a[j'] < a[i] + a[j] < X.$$

So, answer cannot contain i and we can increase i .

Similarly, if $a[i] + a[j] > X$, we can get rid of j :

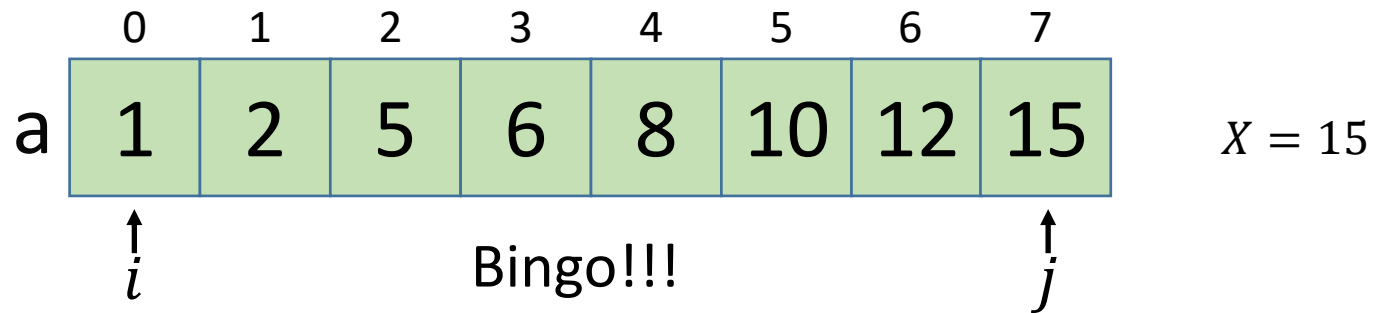
$$\forall i' \in [i, j]: a[i'] + a[j] > a[i] + a[j] > X.$$

So, answer cannot contain j and we can decrease j .

Finally, we either found desired pair, or $i = j$, this means that desired pair does not exist.

Linear algorithms. Two pointers

Pair of values with given sum

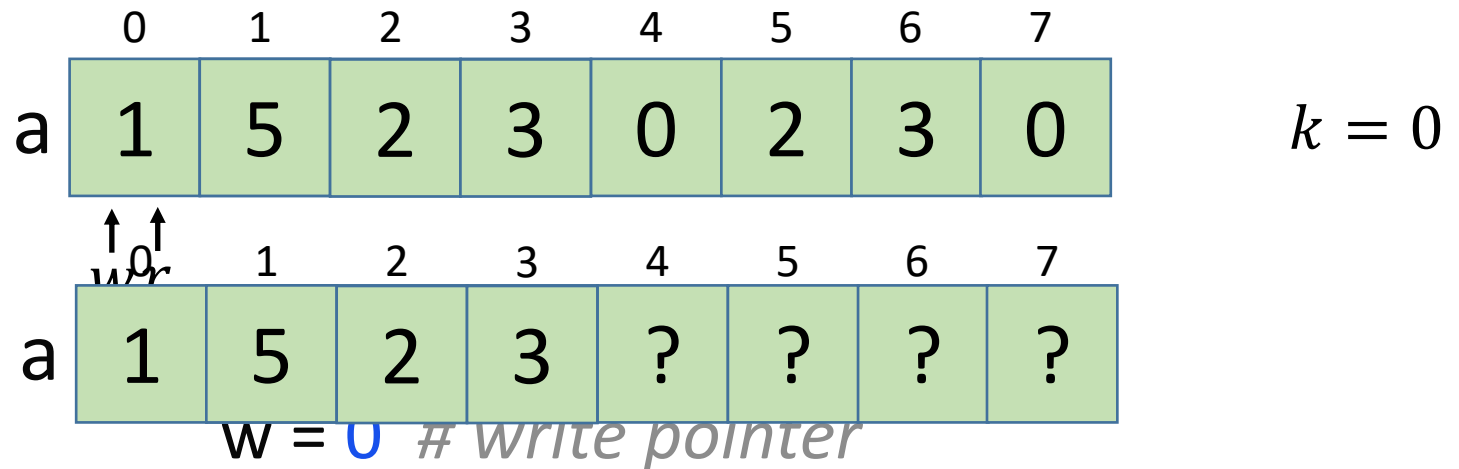


```
i = 0
j = N - 1
while i != j:
    if a[i] + a[j] < X:
        i += 1
    elif a[i] + a[j] > X:
        j -= 1
    else: # a[i] + a[j] == X
        return i, j
return None
```

Complexity: $O(N)$.

Linear algorithms. Two pointers

Remove all k values



```
for r in range(len(a)): # read pointer
    if a[r] != 0:
        a[w] = a[r]
        w += 1
```

Complexity: $O(N)$.

Thank you for watching!